

# Lesson 2: HTML Structure and Semantic Elements

## Mission Brief

A ship's blueprint isn't just a pile of walls — it labels the bridge, the engine room, and the cargo bay so every crew member knows what's what. Your HTML pages need the same clarity. Today, Navigator, you'll upgrade your first page from generic boxes into a properly labeled starship layout using **semantic elements** — tags that describe *meaning*, not just appearance.

## Learning Objectives

- Distinguish semantic elements from generic containers ( `<div>` / `<span>` )
- Correctly structure a page using `<header>` , `<nav>` , `<main>` , `<section>` , `<article>` , `<aside>` , and `<footer>`
- Use heading levels ( `<h1>` - `<h6>` ) in a logical, non-skipping order
- Build lists, links, and images with correct attributes
- Explain why semantic HTML matters for accessibility and SEO

## What You'll Build

A structured single-page "Crew Profile" document using proper semantic sectioning, a navigation menu, an image with alt text, and a list — the direct ancestor of your Project 01 personal landing page.

## Prerequisites

Lesson 1: a working `<!DOCTYPE html>` skeleton, and how to open/view HTML files in a browser.

## Key Concepts

- Semantic HTML
- `<header>` , `<nav>` , `<main>` , `<section>` , `<article>` , `<aside>` , `<footer>`
- Heading hierarchy
- `<a>` (anchor/link) and `href`
- `<img>` and `alt` text
- `<ul>` , `<ol>` , `<li>`
- Attributes vs. elements

- Accessibility (a11y) basics

## Concept Explanation

Early HTML pages (and many beginner pages today) are built almost entirely out of `<div>` tags — generic, meaningless boxes. They work visually, but they tell a browser, a search engine, or a screen reader nothing about what each box *is*. HTML5 introduced **semantic elements**: tags whose names describe their purpose. A `<nav>` is understood by browsers and assistive technology to be navigation. A `<footer>` is understood to be the closing section of a page. This matters for three big reasons: **accessibility** (screen readers announce landmarks like "navigation" or "main content" so blind users can jump straight to them), **SEO** (search engines weight content inside `<article>` or proper headings more meaningfully than an unlabeled `<div>`), and **maintainability** (a developer — including future you — can scan the tag names and instantly understand the page's layout).

The typical semantic skeleton of a page looks like this: a `<header>` at the top (often containing a logo and `<nav>`), a `<main>` element that wraps the single most important content of the page (there should only be one `<main>` per page), broken into `<section>` elements for distinct thematic groups (like "About," "Skills," "Contact"), and a `<footer>` at the bottom for copyright, social links, or site-wide info. `<article>` is for content that could stand alone and make sense if syndicated elsewhere, like a blog post or a product card. `<aside>` is for tangentially related content, like a sidebar.

Headings ( `<h1>` through `<h6>` ) form an **outline** of your page, similar to a table of contents. There should be exactly one `<h1>` per page — it's the main title — and subsequent headings should nest logically without skipping levels (don't jump from `<h1>` straight to `<h4>` ).

Links use the `<a>` (anchor) tag with an `href` attribute pointing to a destination — another page, another site, or an anchor within the same page ( `href="#contact"` ). Images use the self-closing `<img>` tag with a required `src` (the image path or URL) and an equally important `alt` attribute — descriptive text shown if the image fails to load and read aloud by screen readers. An `<img>` without meaningful `alt` text is a common and serious accessibility failure.

Attributes, in general, are extra information attached to the *opening* tag of an element, written as `name="value"` pairs — they modify or configure the element without becoming visible content themselves (unlike the text between opening and closing tags).

## Guided Coding

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Crew Profile – Nova Ortiz</title>
  </head>
  <body>
    <!-- HEADER: site identity and primary navigation -->
    <header>
      <h1>Nova Ortiz – Cadet Engineer</h1>
      <nav>
        <ul>
          <li><a href="#about">About</a></li>
          <li><a href="#skills">Skills</a></li>
          <li><a href="#contact">Contact</a></li>
        </ul>
      </nav>
    </header>

    <!-- MAIN: the single most important content region of the page -->
    <main>
      <section id="about">
        <h2>About Me</h2>
        <p>
          I'm a cadet engineer training in the Lumexa fleet, focused on
          building sturdy, well-structured web pages before adding style.
        </p>
        
      </section>

      <section id="skills">
        <h2>Skills in Progress</h2>
        <ul>
          <li>Semantic HTML structure</li>
          <li>Writing accessible alt text</li>
          <li>Building navigation menus</li>
        </ul>
      </section>

      <aside>
        <h2>Fun Fact</h2>
        <p>My first line of code was a semantic HTML page, just like this one!</p>
      </aside>
    </main>

```

```

    <!-- FOOTER: closing, site-wide information -->
    <footer id="contact">
      <p>&copy; 2026 Nova Ortiz. Reach me at nova@example.com</p>
    </footer>
  </body>
</html>

```

## Code Walkthrough

- `<header>` wraps the page title and the `<nav>` — this is the top landmark region.
- `<nav>` contains a `<ul>` list of `<a>` links; each `href="#about"` jumps to the element with `id="about"` elsewhere on the page — this is called a same-page anchor link.
- `<main>` wraps everything that is the unique content of this page — only one `<main>` should exist per document.
- Each `<section>` groups one topic and starts with its own `<h2>`, keeping the heading hierarchy under the page's single `<h1>`.
- `<img>` has both `src` (where the image file is) and `alt` (a text description) — never omit `alt`.
- `<aside>` holds content related to but separate from the main flow — here, a tangential fun fact.
- `<footer>` closes the page, and its `id="contact"` is what the nav's "Contact" link jumps to.
- `&copy;` is an HTML **entity** — a special code for a character (©) that could otherwise be tricky to type directly.

## Student Mission

1. Build the "Crew Profile" page above with your own name and content.
2. Add a fourth navigation link and a matching fourth `<section>` (e.g. "Hobbies" or "Goals").
3. Add at least one ordered list ( `<ol>` ) somewhere on the page (e.g. "My Top 3 Learning Goals, in priority order").
4. Make sure every heading level you use is in correct order (no skipping from `<h1>` to `<h3>` ).

## Challenge

Convert one of your `<section>` blocks into an `<article>` instead, and explain in a code comment why that content could reasonably "stand alone." Then add a second `<img>` with deliberately empty `alt=""` for an image that is purely decorative, and explain in a comment why an empty alt (rather than missing alt) is the correct choice for decorative images.

## Experiment/Extension

Try removing all the semantic tags ( `<header>` , `<nav>` , `<main>` , `<footer>` ) and replacing them with plain `<div>` tags with matching class names. Visually, nothing changes in the browser. Now open DevTools → Elements and notice there's no "Accessibility" landmark information the same way. This shows semantic tags carry *meaning*, not just layout.

## Common Mistakes

- Using more than one `<h1>` per page.
- Skipping heading levels (e.g. `<h2>` straight to `<h4>` ).
- Leaving `alt` attributes empty on meaningful images (or omitting them entirely).
- Using `<div>` for everything instead of semantic tags where one clearly fits.
- Forgetting that `href="#id"` requires an element elsewhere with exactly that `id` (case-sensitive, no `#` in the id itself).

## Debugging Tips

- In DevTools, the **Elements** panel's "Accessibility" pane (in the sidebar) shows the accessibility tree — use it to confirm your landmarks ( `header` , `nav` , `main` , `footer` ) are recognized.
- If a same-page link doesn't scroll anywhere, check that the target `id` matches exactly (typos and case matter).
- Use the browser's "View Page Source" (Ctrl+U / Cmd+Option+U) to see your raw HTML exactly as the server would send it, useful for catching stray unclosed tags.

## Check Your Understanding

1. What's the difference between `<section>` and `<article>` ?
2. Why should there be only one `<h1>` on a page?
3. What two attributes are essential on every meaningful `<img>` ?
4. What does `href="#contact"` do?
5. Name one accessibility benefit and one SEO benefit of semantic HTML.

## Mini Quiz + Answer Key

**Quiz** 1. Which tag is meant to wrap the single unique content of a page? a) `<div>` b) `<main>` c) `<body>` d) `<section>` 2. True or False: `alt=""` is always wrong. 3. What list tag would you use for numbered, ordered steps? 4. Which tag is best for content unrelated to the main flow, like a sidebar tip? 5. What attribute makes an `<a>` tag actually link somewhere?

**Answer Key** 1. b) `<main>` 2. False — empty alt is correct for purely decorative images. 3. `<ol>` 4. `<aside>` 5. `href`

## Lesson Recap

- Semantic elements describe meaning, not just appearance.
- `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>` are the core layout landmarks.
- Headings must form a logical, non-skipping outline.
- Links need `href`; images need `src` and `alt`.
- Semantic structure improves accessibility, SEO, and code readability.

## Homework

Rebuild your `mission-log.html` from Lesson 1 using at least four different semantic elements from this lesson, keeping all original content but reorganizing it properly.

## Portfolio Project Connection

This lesson's semantic skeleton ( `header` → `nav` → `main` with `sections` → `footer` ) becomes the literal structural backbone of **Project 01 (Personal landing page)** and the shared page structure reused across **Project 03 (Product showcase site)**'s multiple pages.